Практическая работа №3

Форматы данных

Цель работы: на практике изучить текстовые и бинарные форматы сериализации данных (JSON, XML, YAML, MessagePack, CBOR, Protobuf), сравнить их размеры, скорость и читаемость; освоить передачу данных по сети (TCP) и понять принципы схемной эволюции.

1. Ответы на вопросы

Чем отличаются текстовые и бинарные форматы?

Текстовые форматы (JSON, XML, YAML) читаются человеком и удобны для отладки, но избыточны: имена полей, кавычки, скобки и теги занимают много места, а разбор идёт медленнее. Бинарные форматы (MessagePack, CBOR, Protobuf) хранят данные в компактном двоичном виде, поэтому они меньше и быстрее, но без специальных инструментов их невозможно прочитать глазами.

Что такое схемная эволюция?

Схемная эволюция — это способность добавлять или удалять поля в структуре данных без поломки работы старых систем. В Protobuf поля идентифицируются номерами, поэтому новый код читает старые данные (новое поле получает значение по умолчанию), а старый код спокойно пропускает неизвестные ему поля.

2. Результаты выполнения шагов

Структура проекта

```
date_format/
├── data.py            # исходные данные
├── profile.proto      # схема Protobuf
├── profile_pb2.py     # сгенерирован protoc
├── step_json.py   step_xml.py   step_yaml.py
├── step_msgpack.py    step_cbor.py   step_protobuf.py
├── server.py          # TCP-сервер
├── client.py          # TCP-клиент
├── check_evolution.py # проверка схемной эволюции
└── run_all.py         # сводный отчёт
```

Таблица 1. Сравнение текстовых форматов

Формат	Размер (байт)	Читаемость (1-5)	Есть ли комментарии?
JSON	253	5	Нет
XML	283	3	Да (<!-- -->)
YAML	182	5	Да (#)

Размер JSON указан для вывода step_json.py (indent=2); в сводной таблице используется компактный JSON.

Таблица 2. Сводная таблица всех форматов (вывод run_all.py)

Формат	Размер (байт)	Читаем ли человеком?	Нужна ли схема?
JSON	211	Да	Нет
XML	283	Да	Нет
YAML	182	Да	Нет
MessagePack	157	Нет	Нет
CBOR	159	Нет	Нет
Protobuf	117	Нет	Да

Скриншоты

Скриншот 1. Вывод консоли step_json.py

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ python step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна Смирнова
```

Скриншот 1. Вывод консоли step_protobuf.py (продолжение)

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ python step_protobuf.py
=== Protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b
0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160
a0cd09cd0bed181d0bad0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
```

Скриншот 2. Работа сервера, формат JSON (TEXT читаемый)

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ python server.py   # во втором окне: python client.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 52814)
=== Получено 211 байт, формат: json ===
HEX: 7b226964223a2034322c20226e616d65223a2022d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0222
c2022616765223a2032322c202269735f73747564656e74223a20747275652c2022636f7572736573223a205b22d09cd0b
0d182d0b5d0bcd0b0d182d0b8d0bad0b0222c2022d0a4d0b8d0b7d0b8d0bad0b0222c2022d098d0bdd184d0bed180d0bcd
0b0d182d0b8d0bad0b0225d2c2022616464726573732223a207b2263697479223a2022d09cd0bed181d0bad0b2d0b0222c2
0227a6970223a20223130313030227d7d
TEXT: {"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика",
 "Физика", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
DATA: {'id': 42, 'name': 'Анна Смирнова', 'age': 22, 'is_student': True, 'courses': ['Математика',
 'Физика', 'Информатика'], 'address': {'city': 'Москва', 'zip': '101000'}}
[СОХРАНЕНО] received/message_json.json (211 байт)
```

Скриншот 2. Работа сервера, формат Protobuf (TEXT не текст)

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ python server.py   # во втором окне: python client.py
Сервер слушает 127.0.0.1:9999...
Подключился: ('127.0.0.1', 52814)
=== Получено 117 байт, формат: protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012a14d09cd0b0d182d0b5d0bcd0b
0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160
a0cd09cd0bed181d0bad0b2d0b01206313031303030
TEXT: <не текст, бинарные данные>
DATA: {'id': 42, 'name': 'Анна Смирнова', 'courses': ['Математика', 'Физика', 'Информатика']}
[СОХРАНЕНО] received/message_protobuf.pb (117 байт)
```

Скриншот 3. Папка received/

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ ls -l received/ && cat received/message_json.json
message_cbor.cbor
message_json.json
message_msgpack.msgpack
message_protobuf.pb
report.json
{"id": 42, "name": "Анна Смирнова", "age": 22, "is_student": true, "courses": ["Математика",
"Физика", "Информатика"], "address": {"city": "Москва", "zip": "101000"}}
(.json открывается как текст; .pb и .msgpack — нет)
```

Скриншот 4. Проверка схемной эволюции (в profile.proto добавлено string email = 7;)

```
User@DESKTOP-RQJBQPN MINGW64 ~/Desktop/date_format (main)
$ protoc --python_out=. profile.proto && python check_evolution.py
Имя: Анна Смирнова
Email (новое поле): ''
```

Вывод: новый код прочитал старые данные без ошибок — это обратная совместимость.

3. Выводы

1. Почему Protobuf и MessagePack оказались меньше JSON?

В бинарных форматах нет кавычек, скобок, запятых и пробелов: числа и длины строк записываются в двоичном виде в один-два байта. MessagePack всё равно хранит имена полей, поэтому он занимает 157 байт против 211 у JSON. Protobuf вместо имён полей записывает их номера из схемы, поэтому он самый компактный — 117 байт.

2. В каком формате данные было легче всего читать вручную?

Легче всего читать YAML и JSON. YAML выглядит чище за счёт отступов и отсутствия кавычек и скобок, JSON привычнее и однозначнее. XML читается хуже из-за парных тегов, а бинарные форматы без инструментов видны только как HEX.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

В JSON нет формальной схемы: структура задаётся самими именами ключей, и изменение набора полей приходится согласовывать между отправителем и получателем вручную, иначе возможны ошибки. В Protobuf есть явная схема, а каждое поле имеет постоянный номер. Старый код пропускает незнакомые номера, а новый код подставляет значения по умолчанию для отсутствующих полей, поэтому старые и новые версии работают вместе.

Чек-лист

[x] все .py и .proto файлы созданы [x] сгенерирован profile_pb2.py [x] скриншоты вывода [x] таблицы заполнены [x] выводы обоснованы